

of your target platform into the Eclipse IDE. If your target platform is AVR, you can use Eclipse-AVR plugins, or if your target platform is ARM, you can use GNU ARM Eclipse plugin. So the developing environment remains the same for text editor, compilation and debugging tools.”

Karthick N.G., technical lead, Honeywell, shares, “Eclipse has a very powerful code editor, and it can be easily configured to use different build tools. It has tools for refactoring code and easy collaboration. Also, it is good for assigning coding tasks and tracking. One great feature is the TODO list, which you can create by adding ‘TODO:’ in your comments. Another is that compiler errors are shown, as and when you type code, by a glaring red underline. This helps to write correct syntax first time. It also features auto-complete, which is pretty standard nowadays. It is easily linkable with version control systems such as SVN and CVS.”

There is always a flip side to any technology. Pubudu Karunaratna, senior electronics engineer at Zone24x7, says, “Eclipse can be somewhat overweight sometimes. Since it is quite large, it can be slow on constrained systems. Also, when you use SVN with Eclipse (with Subclipse plugin), the SVN commands in Windows Explorer do not function well. Specifically, you do not find modified files shown in Explorer.”

Other popular and OS-compatible platforms

The choice of the tool chain depends on the embedded product. Many chip companies have extended their support in development of open source tools for embedded applications. There are open source tools and extensive support available for ARM, MSP430, AVR, etc.

“AVR Studio IDE (free) uses WINAVR package (free) for compiling the source programs, which supports many microcontrollers from ATMEL. In AVR Studio IDE, you can simulate

Popular OSS tools for embedded developers

- U-Boot Bootloader
- Linux Kernel
- Android
- GCC, Binutils, Busybox
- GDB
- OpenGL ES library
- OpenVG library
- OpenSSL
- WebKit
- JFFS2, UBIFS, BTRFS and other file systems
- Ubuntu, Fedora Core and other file system distributions for ARM
- G-Streamer
- QT
- VLC (VideoLAN)
- FFMpeg
- OpenOCD (open on-chip debugger)
- GDB debugger and its front-end DDD
- Version control tools like CVS and SVN
- Various GNU bin-utilities like profilers and converters
- Cygwin
- WINAVR
- GNUARM
- Yagarto

your programs. You can also debug your programs using JTAG-ICE, which is run-time debugging where you can set the hardware breakpoints and continue executing till the breakpoint and also keep a watch at the variables, memory layouts, etc,” shares Shah.

“Code Composer Studio Version 4 is based on open source Eclipse frameworks, and there is an extensive community support for the same. For mobile development, Qt Creator is much easier to use. It is available in both Windows and Linux,” informs Karunaratna.

Open source challenges in embedded

An embedded system comprises hardware, interfacing software and application software. In application software, open source platform has achieved certain maturity level. But there are instances in embedded projects when designers had to opt for proprietary

software because of the lack of support, inability to perform and untimely support. Availability of open source is also a challenge in certain embedded projects.

“For certain mission-critical applications, such as defence electronics and avionics, proprietary tools are preferred,” says Vasa. There are several versions co-existing, and not fully tested, which makes open source difficult to use in critical applications. “Lack of future roadmap, feature support and change in end-user license agreement (EULA) during project life-cycle are some of the other reasons that are not in favour of open source for such applications.”

For programmable hardware like FPGAs. There are no open source tools for components like field-programmable gate arrays (FPGAs). Karunaratna says, “If your design has FPGAs like Xilinx Spartan-x family or Virtex family, you will have to go with Xilinx ISE. The free Web edition of Xilinx supports only a certain number of devices.”

Nair explains, “Designing with FPGAs is hardware design. It involves knowing about delays, clock rates, clock skews, current sourcing and sinking, and a lot of other stuff that only the guys who designed the FPGA know about. Hence I suspect the lack of open source design tools for the FPGAs on the market. There are some open cores, but if you design with them, you are stuck with having to fabricate them.”

Sometimes, it's performance. Open source tools like gcc are multi-platform tools. Hence the level at which the code is optimised for each platform is a question. Walia says, “Sometimes a few specifications are required to be overly optimised, in which case you have to move to proprietary tools for better support.”

Vasa adds, “For this system-critical performance, optimised codecs for different profiles are necessary.”

Jain informs, “In one of the instances, we had to reduce our code size to fit into a particular microcontroller.